

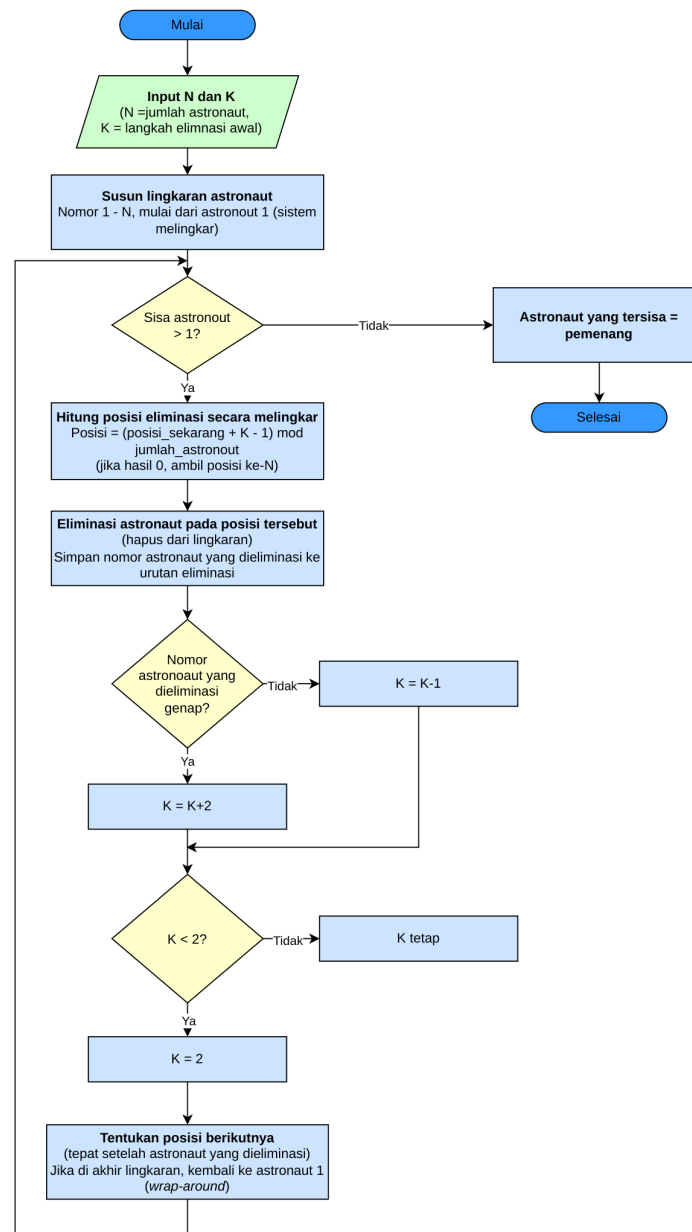
FASE 1

Nama: Katon Dian Candra | NIM: 26/576803/TK/65606

Kelompok: Topaz – Prodi: Teknik Elektro

PROBLEM 1: THE LAST ASTRONAUT

Flowchart:



Penjelasan Logika Algoritma (Problem 1)

Problem 1 memodelkan variasi permasalahan klasik *Josephus Problem* untuk menyeleksi astronot dalam formasi melingkar secara bertahap hingga menyisakan satu orang pemenang (*the last astronaut*).

Alur logika pada diagram alir (*flowchart*) terbagi ke dalam tahapan-tahapan berikut:

1. Inisialisasi dan Pembentukan Formasi Lingkaran

- Alur program dimulai dari blok terminal **Mulai**, kemudian menerima masukan berupa jumlah total astronot (N) dan nilai langkah hitungan awal (K).
- Sistem menyusun lingkaran astronot dengan nomor unik 1 hingga N yang terurut melingkar, dengan penghitungan awal dimulai tepat dari astronot nomor 1.

2. Perulangan Kondisi Eliminasi dan Perhitungan Posisi

- Program mengevaluasi kondisi terminasi melalui percabangan belah ketupat **Apakah sisa astronot > 1?**.
- Selama kondisi bernilai "Ya", proses eliminasi dijalankan dengan menghitung posisi astronot yang gugur menggunakan aritmetika modular:
$$\text{Posisi} = (\text{posisi_sekarang} + K - 1) \pmod{\text{jumlah_astronot}}$$

Perhitungan modular ini secara otomatis menangani perpindahan melingkar (*wrap-around*) jika hitungan langkah melampaui batas akhir lingkaran.
- Astronot yang berada pada posisi target tersebut dieliminasi (dikeluarkan dari lingkaran) dan nomor uniknya dicatat ke dalam daftar urutan eliminasi.

3. Pembaruan Adaptif Nilai Langkah K

Setelah terjadi eliminasi, alur memeriksa paritas dari nomor astronot yang baru saja gugur melalui percabangan **Apakah nomor astronot yang dieliminasi genap?**:

- **Jika bernilai "Ya" (Genap):** Nilai langkah diperbesar dengan penambahan dua ($K = K + 2$).
- **Jika bernilai "Tidak" (Ganjil):** Nilai langkah diperkecil dengan pengurangan satu ($K = K - 1$).

Selanjutnya, sistem memvalidasi batas bawah melalui percabangan **Apakah $K < 2$?**:

- **Jika $K < 2$ (Ya):** Nilai K dipatok kembali menjadi nilai minimum aman, yaitu $K = 2$, guna mencegah perputaran nol atau nilai negatif.
- **Jika tidak ($K \geq 2$):** Nilai K dipertahankan sesuai hasil perhitungan sebelumnya.

4. Penentuan Titik Awal Putaran Baru

- Sistem menentukan titik awal hitungan berikutnya tepat dari astronot yang berada setelah astronot yang dieliminasi.
- Jika posisi eliminasi berada di ujung lingkaran, hitungan putaran baru otomatis berputar kembali (*wrap-around*) ke astronot pertama di awal barisan lingkaran.
- Garis alur kemudian kembali mengarah ke atas menuju pemeriksaan kondisi **Sisa astronot > 1?** untuk melanjutkan putaran berikutnya.

5. Terminasi dan Penetapan Pemenang

- Perulangan berhenti ketika kondisi **Sisa astronot > 1?** menghasilkan nilai "Tidak", yang menandakan seluruh putaran eliminasi selesai dan hanya tersisa tepat satu astronot di dalam formasi lingkaran.
- Program mengumumkan satu astronot yang tersisa tersebut sebagai pemenang (*survivor*) beserta seluruh riwayat urutan eliminasi, sebelum akhirnya mencapai blok terminal **Selesai**

PROBLEM 2: ALIEN-IN-THE-MIDDLE

Pseudocode:

```
ALGORITMA Alien_In_The_Middle
{Sistem sandi rahasia: Enkripsi pesan asli dan Dekripsi
kembali untuk alien}

DEKLARASI:
    pesanAsli, pesanSandi, pesanDekripsi : ARRAY OF CHAR
    panjang, i                             : INTEGER
    valCurrent, valPrev, newVal            : INTEGER
    valCipher, valPrevPlain, origVal       : INTEGER
    selisih                                : INTEGER
    prevChar, prevPlain                    : CHAR

DESKRIPSI:
    INPUT(pesanAsli)
    panjang <- hitungPanjang(pesanAsli)

    // =====
    // PROSES 1: ENKRIPSI (Pesan Asli -> Sandi)
    // =====
    // Huruf pertama tidak mengalami perubahan
    pesanSandi[0] <- ubahKeKapital(pesanAsli[0])
    prevChar      <- pesanSandi[0]

    // Karakter berikutnya digeser sebesar nilai huruf
    asli sebelumnya
    UNTUK i <- 1 SAMPAI (panjang - 1) LAKUKAN:
        IF apakahAlfabet(pesanAsli[i]) THEN
            valCurrent                                     <-
nilaiAlfabet(ubahKeKapital(pesanAsli[i]))
            valPrev      <- nilaiAlfabet(prevChar)

            // Pergeseran melingkar modulo 26 (A=1 ..
Z=26)
            newVal <- ((valCurrent - 1 + valPrev) MOD 26)
+ 1
            pesanSandi[i] <- hurufDariNilai(newVal)

            // Simpan huruf asli saat ini sebagai acuan
langkah berikutnya
            prevChar <- ubahKeKapital(pesanAsli[i])
        ELSE
            pesanSandi[i] <- pesanAsli[i]
```

```

        ENDIF
    AKHIR UNTUK
    OUTPUT("Hasil Enkripsi: ", pesanSandi)

    // =====
    // PROSES 2: DEKRIPSI (Sandi -> Pesan Asli)
    // =====
    // Huruf pertama sandi sama dengan pesan asli
    pesanDekripsi[0] <- pesanSandi[0]
    prevPlain        <- pesanDekripsi[0]

    UNTUK i <- 1 SAMPAI (panjang - 1) LAKUKAN:
        IF apakahAlfabet(pesanSandi[i]) THEN
            valCipher      <- nilaiAlfabet(pesanSandi[i])
            valPrevPlain   <- nilaiAlfabet(prevPlain)

            // Pembalikan geseran (dekripsi)
            selisih <- (valCipher - 1 - valPrevPlain) MOD
26
            IF selisih < 0 THEN
                selisih <- selisih + 26
            ENDIF
            origVal <- selisih + 1
            pesanDekripsi[i] <- hurufDariNilai(origVal)

            // Perbarui acuan huruf asli untuk karakter
berikutnya
            prevPlain <- pesanDekripsi[i]
        ELSE
            pesanDekripsi[i] <- pesanSandi[i]
        ENDIF
    AKHIR UNTUK
    OUTPUT("Hasil Dekripsi Kembali: ", pesanDekripsi)

```

Penjelasan Logika Algoritma (Problem 2):

Problem 2 mengimplementasikan sistem kriptografi simetris berbasis substitusi polialfabetik diferensial berantai. Berbeda dari sandi Caesar konvensional yang menggeser karakter menggunakan kunci statis, sistem ini memanfaatkan nilai ordinal karakter tepat sebelumnya pada pesan asli sebagai kunci pergeseran dinamis.

Alur logika dan mekanisme algoritma terbagi ke dalam beberapa poin penting:

1. Karakter Pertama sebagai Pasak Utama (*Initial Anchor Point*)

- Karakter pertama dari pesan asli tidak mengalami pergeseran sama sekali dan langsung disalin menjadi karakter pertama sandi ($C[0] = P[0]$).
- Karakter pertama yang tidak dimodifikasi ini bertindak sebagai pasak atau jangkar penentu (*anchor point*). Keberadaannya sangat krusial karena nilai ordinal huruf asli pertama akan menjadi kunci pergeseran bagi huruf kedua, nilai huruf asli kedua menjadi kunci pergeseran bagi huruf ketiga, dan seterusnya secara berantai.

2. Formulasi Pergeseran dan Aritmetika Modulo (Enkripsi)

- Setiap alfabet dipetakan ke dalam nilai ordinal standar, yaitu A = 1 hingga Z = 26.
- Ketika penjumlahan nilai huruf saat ini dan huruf sebelumnya melebihi 26, sistem memutar kembali urutannya ke awal alfabet (A).
- Perputaran melingkar ini dimodelkan secara matematis menggunakan aritmetika modulo:
$$\text{nilaiBaru} = ((\text{valCurrent} - 1 + \text{valPrev}) \bmod 26) + 1$$
- Pengurangan angka 1 di awal berfungsi memetakan domain nilai 1–26 ke indeks modular 0–25 agar operasi sisa bagi (modulo 26) bekerja konsisten (misalnya nilai 26 menjadi 25, dan kelipatan 26 menjadi 0). Penambahan angka 1 di akhir mengembalikan hasilnya ke rentang alfabet 1–26.
- Sebagai contoh pada kata ALIENS: huruf 'S' (19) yang didahului huruf 'N' (14) menghasilkan nilai 33. Melalui operasi perputaran, $((33 - 1) \bmod 26) + 1$ menghasilkan 7, yang bersesuaian dengan huruf 'G'.

3. Rekonstruksi Pesan Dua Arah secara Induktif (Dekripsi)

- Agar sang alien dapat memahami pesan sandi tersebut, algoritma dirancang bersifat reversibel (*invertible*).
- Karena kunci pergeseran bertumpu pada teks asli sebelumnya (*forward dependency*), proses dekripsi dapat dipecahkan langkah demi langkah secara induktif:

1. Huruf asli pertama langsung diketahui dari karakter pertama sandi ($P[0] = C[0]$).

2. Huruf asli berikutnya dipulihkan dengan membalikkan operasi pergeseran:

$selisih = (valCipher - 1 - valPrevPlain) \bmod 26$

(Jika selisih bernilai negatif, ditambahkan 26 agar kembali ke siklus positif)

$origVal = selisih + 1$

3. Huruf asli yang baru terungkap langsung dijadikan acuan kunci untuk mendekripsi huruf berikutnya hingga seluruh kalimat kembali utuh.